

Payment Voucher "v2" (Telerik) — Code Walkthrough

What this covers: every file that participates when you click "معاينة بيان الدفعة" (Telerik) on a payment, traced the way a debugger would step through it — frame by frame, which line hands off to which file, and what's in the key variables at each hop. Plus a section on the config/infra files that were changed to make Telerik Reporting work at all.

The request being traced (from the server log):

HTTP GET /api/paymentReports/payment-report-v2/018037 → 200 (PDF bytes)

018037 is a disbursement payment (`PaymentParentTypeDescription != "RECEIPT"` → إيصال صرف).

1. The cast — files involved

#	File	Layer	Role in this request
F1	<code>client-app/.../payment/form/EditPaymentForm.tsx</code>	React UI	The button, the click handler, the PDF preview dialog
F2	<code>client-app/src/app/store/apis/payment/paymentsApi.ts</code>	RTK Query	Defines the <code>getPaymentReportPdfV2</code> endpoint + generated hook; attaches auth + <code>Accept-Language</code>
F3	<code>API/Controllers/BaseApiController.cs</code>	API	Base class: route prefix <code>api/[controller]</code> , lazy <code>Mediator</code> accessor
F4	<code>API/Controllers/Report/PaymentReports/PaymentReportsController.cs</code>	API	<code>GetPaymentReportV2</code> action — orchestrates data → render → file
F5	<code>API/Extensions/ApplicationServiceExtensions.cs</code>	API composition root	DI: binds <code>IPaymentVoucherReportService</code> → <code>TelerikPaymentVoucherReportService</code> ; registers MediatR + pipeline behaviors
F6	<code>Application/Accounting/Payments/GeneratePaymentReportPdf.cs</code>	Application (CQRS)	<code>GetPaymentForReport.Query</code> + <code>.Handler</code> + <code>PaymentReportDto</code> — the one DB read
F7	<code>Application/Interfaces/IPaymentVoucherReportService.cs</code>	Application (port)	The contract the controller depends on (keeps API → Telerik decoupled at the seam)
F8	<code>API/Reporting/TelerikPaymentVoucherReportService.cs</code>	API (adapter)	Wraps <code>ReportProcessor</code> ; validates format; turns a <code>PaymentReportDto</code> into <code>byte[]</code>
F9	<code>API/Reporting/PaymentVoucherReport.cs</code>	API (report definition)	Code-defined <code>Telerik.Reporting.Report</code> — builds the voucher layout item by item
F10	<code>Application/Accounting/Payments/ArabicPaymentFormatter.cs</code>	Application (helper)	Amount-in-words, Arabic-Indic digits, currency suffix, date stencil
F11	<code>Infrastructure/Pdf/PdfGenerationService.cs</code>	Infrastructure	Not called here — the v1 (QuestPDF) engine, shown for contrast

Everything F6–F10 is also reachable from the `v1` endpoint except F8/F9; both endpoints share F6 (the data) and F10 (the Arabic helpers).

2. Debugger walkthrough — frontend

STEP 1 — `EditPaymentForm.tsx:888` — the click

```
<Button variant="outlined" color="secondary"
  onClick={handlePreviewPdfV2}
  disabled={isPdfV2Fetching || !payment?.paymentId}>
  {isPdfV2Fetching ? "معاينة بيان الدفعة" : "جاري التحضير..."} (Telerik)
</Button>
```

`payment` is the currently-loaded payment object (form state). Click → jump to `handlePreviewPdfV2` .

STEP 2 — `EditPaymentForm.tsx:120` — the handler

```
const handlePreviewPdfV2 = async () => {
  if (!payment?.paymentId) return;           // guard
  setPreviewLoading(true);                   // dialog spinner on
  try {
    const arrayBuffer = await triggerPdfV2(payment.paymentId).unwrap(); // ← line 125
    const blob = new Blob([arrayBuffer], { type: "application/pdf" });
    const url = URL.createObjectURL(blob);
    setPdfBlobUrl(url);
    setShowPdfViewer(true);
  } catch (error) { /* alert */ } finally { setPreviewLoading(false); }
};
```

- `triggerPdfV2` came from `EditPaymentForm.tsx:148`: `const [triggerPdfV2, { isFetching: isPdfV2Fetching }] = useLazyGetPaymentReportPdfV2Query();`
- Calling `triggerPdfV2("018037")` jumps into RTK Query's generated `thunk` → the endpoint defined in F2.
- Execution now *suspends* at `await` until the HTTP round-trip resolves.

STEP 3 — `paymentsApi.ts:290` — the endpoint definition

```
getPaymentReportPdfV2: builder.query<ArrayBuffer, string>({
  query: (paymentId) => ({
    url: `/paymentReports/payment-report-v2/${paymentId}`,
    responseHandler: (response) => response.arrayBuffer(), // raw bytes, not JSON
    cache: "no-cache",
  }),
}),
```

- Base URL: `paymentsApi.ts:14` → `import.meta.env.VITE_API_URL` → `http://localhost:5001/api` (dev).
- Final URL: `http://localhost:5001/api/paymentReports/payment-report-v2/018037`.
- `paymentsApi.ts:15–24` `prepareHeaders` runs: adds `Authorization: Bearer <token>` and `Accept-Language: <ui language>` from the Redux store.
- `responseHandler` overrides the default JSON parse so the 200 body is read as an `ArrayBuffer`.

→ HTTP request leaves the browser. Next frame is on the server.

3. Debugger walkthrough — ASP.NET request pipeline

STEP 4 — middleware, before the controller

Order (from `API/Program.cs`): Serilog request logging → Response compression → **Authentication** → Routing → **Authorization**.

- `Program.cs` registers a **global** `AuthorizeFilter` (`RequireAuthenticatedUser`), so the bearer token from STEP 3 is mandatory — a missing/expired token would 401 here and never reach F4.
- Routing matches `api/PaymentReports/payment-report-v2/{paymentId}` to F4's action (case-insensitive; the URL said `paymentReports`).

STEP 5 — `PaymentReportsController` construction (DI)

Before the action runs, the container builds the controller. Its constructor (`PaymentReportsController.cs:13`):

```
public PaymentReportsController(
  IPdfGenerationService pdfService,           // v1 (QuestPDF) – not used this request
  IPaymentVoucherReportService voucherReportService) // v2 (Telerik)
```

- `IPaymentVoucherReportService` is resolved because of `ApplicationServiceExtensions.cs:91`: `services.AddScoped<IPaymentVoucherReportService, TelerikPaymentVoucherReportService>();`
- **If that line were missing:** the container throws `"Unable to resolve service for type IPaymentVoucherReportService"` and the request 500s here, before your code.

STEP 6 — `PaymentReportsController.cs:41` — the action

```
[HttpGet("payment-report-v2/{paymentId}")]
public async Task<IActionResult> GetPaymentReportV2(string paymentId, [FromQuery] string format = "PDF")
{
    var reportData = await Mediator.Send(new GetPaymentForReport.Query(paymentId, "ar")); // line 43
    var bytes      = _voucherReportService.Render(reportData, format);                // line 45
    var (contentType, ext) = /* switch on format */ ("application/pdf", "pdf");        // line 47
    var fileName = $"{reportData.PaymentId}_بيان_دفعة_{ext}";                        // line 56
    return File(bytes, contentType, fileName);                                       // line 58
}
```

Locals on entry: `paymentId = "018037"`, `format = "PDF"`.

`Mediator` is the property on `BaseApiController.cs:13`:

```
protected IMediator Mediator => _mediator ??= HttpContext.RequestServices.GetService<IMediator>();
```

Line 43 constructs `new GetPaymentForReport.Query("018037", "ar")` and jumps into `MediatR`.

4. Debugger walkthrough — the data read (CQRS)

STEP 7 — MediatR pipeline behaviors

`Mediator.Send` doesn't call the handler directly. Two behaviors wrap it (`ApplicationServiceExtensions.cs:77` and `:80`):

```
LoggingBehavior<Query,PaymentReportDto>      ← starts a stopwatch
└─ AuditBehavior<Query,PaymentReportDto>      ← writes an AUDIT_ACTIVITY row (commands only; a query is a no-op
here)
    └─ GetPaymentForReport.Handler.Handle(query, ct) ← your code
```

STEP 8 — `GeneratePaymentReportPdf.cs:20` — `GetPaymentForReport.Handler.Handle`

```
public async Task<PaymentReportDto> Handle(Query request, CancellationToken ct)
{
    var isArabic = request.Language.Equals("ar", ...); // true

    var query = from pyt in _context.Payments
                where pyt.PaymentId == request.PaymentId // "018037"
                join ptt in _context.PaymentTypes on ... // INNER
                join sts in _context.StatusItems on ... // INNER
                join pty in _context.Parties on pyt.PartyIdFrom ... // INNER
                join pmt in _context.PaymentMethods ... into pmtJoin from pmt in pmtJoin.DefaultIfEmpty() // LEFT
                join ptyto ... DefaultIfEmpty() // LEFT (PartyIdTo)
                join cc ... DefaultIfEmpty() // LEFT (CostCenter)
                join proj ... DefaultIfEmpty() // LEFT (WorkEffort / project)
                select new PaymentReportDto { /* ~18 fields, Arabic vs English picked by isArabic */ };

    var result = await query.FirstOrDefaultAsync(ct); // line 71 – the SQL executes here
    if (result == null) throw new Exception($"Payment ... not found.");
    return result;
}
```

- `_context` is the `DataContext` injected straight into the handler — **no repository layer** (house rule). Lazy loading is off, so every needed table is an explicit `join`.
- One `SELECT ... LEFT JOIN ...` hits MySQL. EF materialises exactly one `PaymentReportDto` (`GeneratePaymentReportPdf.cs:82`).
- Return value bubbles back out through `AuditBehavior` → `LoggingBehavior` (which logs the handler's elapsed ms) → the `await` on `PaymentReportsController.cs:43`.

State now: `reportData` is a populated `PaymentReportDto` (`PaymentId`, `Amount`, `EffectiveDate`, `FromPartyName`, `ToPartyName`, `PaymentMethodId`, `IsBankTransfer`, `Comments`, ...).

5. Debugger walkthrough — rendering (Telerik)

STEP 9 — `PaymentReportsController.cs:45` → `TelerikPaymentVoucherReportService.cs:20`

```
public byte[] Render(PaymentReportDto data, string format = "PDF", string companyName = "Golden Land")
{
    ArgumentNullException.ThrowIfNull(data);
    var normalizedFormat = "PDF"; // trimmed + upper-cased
    if (!SupportedFormats.Contains(normalizedFormat)) throw ...; // PDF/XLSX/DOCX/CSV/IMAGE/PPTX

    using var report = new PaymentVoucherReport(data, companyName); // ← line 28, jump to F9
    var reportSource = new InstanceReportSource { ReportDocument = report };
    var processor = new ReportProcessor();
    var result = processor.RenderReport(normalizedFormat, reportSource, new Hashtable()); // line 32

    if (result.HasErrors) throw new System.InvalidOperationException(...);
    return result.DocumentBytes; // line 39 – the PDF
}
```

STEP 10 — `PaymentVoucherReport.cs:29` — the report constructor

This is where the layout is assembled. Walking the constructor:

Line	What happens	Notes
31	null-check <code>data</code>	
33–40	derive booleans: <code>isReceipt</code> , <code>isCash</code> , <code>isBankTransfer</code> , <code>isCheque</code>	same rules as the QuestPDF voucher
42	<code>ArabicPaymentFormatter.CurrencySuffix(data.CurrencyUomId)</code>	jump to F10:16 → "جنيه مصري" etc.
43	<code>ArabicPaymentFormatter.AmountToWords(data.Amount, suffix)</code>	jump to F10:45 → recurses through <code>NumberToWords</code> (F10:58) to build "مائتان ... جنيه مصري لا غير"
44–45	split <code>Amount</code> into whole pounds + piastres	
48	<code>Width = Unit.Cm(18.0)</code>	first touch of <code>Telerik.Reporting.Drawing.Unit</code> → its static ctor runs → it loads the drawing backend <code>Telerik.Drawing.Skia</code> . This is the line that threw <code>DrawingFactoryUnavailableException</code> before the <code>Telerik.Drawing.Skia</code> package was added.
49–50	<code>PageSettings.PaperKind = A4</code> , 1.5 cm margins	
52–55	create <code>PageHeaderSection</code> + <code>DetailSection</code> , add to <code>report.Items</code>	a <code>DetailSection</code> with no data source renders exactly once — the trick that makes a "static" report
57	<code>BuildHeader(...)</code>	→ STEP 11
58	<code>BuildBody(...)</code>	→ STEP 12

STEP 11 — `PaymentVoucherReport.cs:61` — `BuildHeader`

- 63–77 load `wwwroot/goldenlandlogo.jpg` and set it via `SKBitmap.Decode(logoPath)`. `PictureBox.Value` accepts `string | IImage | System.Drawing.Image | SKBitmap` — not `byte[]` (that was `ArgumentException: You can assign ...` before the `SKBitmap.Decode` fix).
- 79–89 title (إيصال قيش / إيصال صرف), payment id, 3 company lines — each built by the `Text(...)` factory (`PaymentVoucherReport.cs:210`).
- 91–105 a header rule = an empty `TextBox` with only a bottom border (replaced the `Shape / LineShape` item, whose namespace moved between Reporting versions → `CS0246`).

STEP 12 — `PaymentVoucherReport.cs:108` — `BuildBody`

A single running `y` cursor (cm from the top of the detail section). Each block: `y` is read, one-to-three items are added at that `y`, then `y` is advanced.

```

y=0.2  cbBank / cbCheque / cbCash      (Checkbox factory :250 – a Panel = [X] box + caption)
y=1.4  date line                      "تحريراً في : ٢٠٢٦"
y=2.5  [piastres][جنيه][pounds][قرش] (Boxed factory :240 – full border)
y=3.8  recipient caption + name       (Underlined factory :230 – bottom border only)
y=4.9  amount-in-words                ← the AmountToWords string from STEP 10
y=6.1  نقد / بموجب
y=7.1  مسحوب على بنك
y=8.1  cheque no. + date
y=9.1  تحويل بنكي / اون لاين
y=10.3 + Comments : وذلك عن
y=12.7 signatures block (المستلم / المحاسب / يعتمد + lines)
y=16   مرجع : 018037

```

Factories `Text` / `Underlined` / `Boxed` / `Checkbox` (`PaymentVoucherReport.cs:210–288`) each `new` a `TextBox` (or `Panel`), set `Location` / `Size` in cm, font = `"Arial"` (`FontFamily` const — swap for an Arabic naskh face once the Docker image ships one), and border style. Nothing is data-bound; every `.Value` is a literal string already computed from `data`.

Constructor returns → back to `TelerikPaymentVoucherReportService.cs:29`.

STEP 13 — `TelerikPaymentVoucherReportService.cs:32` — `ReportProcessor.RenderReport`

- Telerik walks the report's item tree, uses **Skia** (via `Telerik.Drawing.Skia`) to measure and shape text (Arabic shaping/bidi happens here) and to rasterize, then emits a PDF stream.
- `Telerik.Licensing` is consulted during processing — because the DevCraft key is present (`~/telerik/telerik-license.txt` locally, `TELERIK_LICENSE` in Docker) there is **no trial watermark**.
- `result.DocumentBytes` → `byte[]` of the finished PDF. Returned to the controller.

6. Debugger walkthrough — response & display

STEP 14 — `PaymentReportsController.cs:47–58`

```

var (contentType, ext) = "PDF" switch { ... _ => ("application/pdf", "pdf") };
var fileName = "018037_بيان_دفعة.pdf";
return File(bytes, "application/pdf", fileName);           // FileContentResult

```

MVC serializes a `200 OK`, `Content-Type: application/pdf`, body = the bytes. Response compression may gzip it on the way out. Serilog logs `responded 200 in <ms>`.

STEP 15 — back in the browser, `EditPaymentForm.tsx:125`

The suspended `await triggerPdfV2(...).unwrap()` now resolves:

```

arrayBuffer ← response.arrayBuffer()      (from paymentsApi.ts responseHandler)
blob        ← new Blob([arrayBuffer], {type:"application/pdf"})
url         ← URL.createObjectURL(blob)   "blob:http://localhost:3000/..."
setPdfBlobUrl(url); setShowPdfViewer(true);

```

The dialog (further down `EditPaymentForm.tsx`) renders `<Viewer fileUrl={pdfBlobUrl} .../>` (`@react-pdf-viewer`) inside a MUI `Dialog`. `handleClosePdfViewer` later calls `URL.revokeObjectURL` to free the blob.

7. One picture — the call sequence



8. v1 vs v2 — same data, different engine

	v1 (existing)	v2 (this walkthrough)
Endpoint	GET /paymentReports/payment-report/{id}	GET /paymentReports/payment-report-v2/{id}?format=PDF
Controller action	GetPaymentReportPdf (F4:22)	GetPaymentReportV2 (F4:41)
Data source	GetPaymentForReport.Query (F6)	same GetPaymentForReport.Query (F6)
Arabic helpers	private methods inside F11	ArabicPaymentFormatter (F10)
Rendering	IPdfGenerationService → QuestPDF (F11)	IPaymentVoucherReportService → Telerik (F8 → F9)
Output	PDF only	PDF / XLSX / DOCX / CSV / IMAGE (via ?format=)
Frontend	getPaymentReportPdf + "معاينة بيان الدفعة (PDF)"	getPaymentReportPdfV2 + "معاينة بيان الدفعة (Telerik)"

The split point is a single DI line (F5:91). The controller depends only on the *interface* (F7), so swapping engines never touches F4's logic.

9. Config / infra files changed for Telerik

9.1 nuget.config — new, repo root

Telerik Reporting packages are **not on nuget.org** — they come from the Telerik private feed.

```

<packageSources>
  <clear />
  <add key="nuget.org" value="https://api.nuget.org/v3/index.json" protocolVersion="3" />
  <add key="telerik" value="https://nuget.telerik.com/v3/index.json" />
</packageSources>
<packageSourceCredentials>
  <telerik>
    <add key="Username" value="api-key" />
    <add key="ClearTextPassword" value="%TELERIK_NUGET_KEY%" /> <!-- from env, never committed -->
  </telerik>
</packageSourceCredentials>
<packageSourceMapping>
  <packageSource key="nuget.org"><package pattern="*" /></packageSource>
  <packageSource key="telerik"><package pattern="Telerik.*" /></packageSource>
</packageSourceMapping>

```

- `packageSourceMapping` keeps every non-Telerik package on nuget.org — a missing key only breaks the `Telerik.*` restore, nothing else.
- Dev must `export TELERIK_NUGET_KEY=<key>` (from telerik.com → account → NuGet keys) before `dotnet restore`.
- **Without this file:** `error NU1101: Unable to find package Telerik.Reporting.`

9.2 API/API.csproj — four package references

```

<PackageReference Include="Telerik.Reporting" Version="20.2.26.812" />
<PackageReference Include="Telerik.Reporting.OpenXmlRendering" Version="20.2.26.812" />
<PackageReference Include="Telerik.Drawing.Skia" Version="20.2.26.812" />
<PackageReference Include="Telerik.Licensing" Version="1.9.2" />

```

Package	Why it's needed
<code>Telerik.Reporting</code>	the report object model + <code>ReportProcessor</code> (F8, F9)
<code>Telerik.Reporting.OpenXmlRendering</code>	the <code>XLSX</code> / <code>DOCX</code> / <code>PPTX</code> render formats (the <code>?format=</code> options)
<code>Telerik.Drawing.Skia</code>	<code>Telerik.Reporting</code> ships only the drawing <i>abstraction</i> ; this is the concrete cross-platform (Skia) implementation. Pulls <code>SkiaSharp 3.119.1</code> — the same version QuestPDF already uses, so no conflict. Without it: <code>DrawingFactoryUnavailableException: Cannot load assembly Telerik.Drawing.Skia</code> on the first <code>Unit.Cm(...)</code> call.
<code>Telerik.Licensing</code>	activates the DevCraft key at build time so output has no trial watermark

9.3 API/Extensions/ApplicationServiceExtensions.cs — the DI binding

```

using API.Reporting; // line 24
...
services.AddScoped<IPaymentVoucherReportService, TelerikPaymentVoucherReportService>(); // line 91

```

The one line that connects the port (F7) to the adapter (F8). **Without it:** the container can't build `PaymentReportsController` and every call to *either* payment-report endpoint 500s (the constructor needs both services).

9.4 Dockerfile.vm — build key, license, fonts

```

# build stage
ARG TELERIK_NUGET_KEY
ARG TELERIK_LICENSE
ENV TELERIK_NUGET_KEY=$TELERIK_NUGET_KEY
ENV TELERIK_LICENSE=$TELERIK_LICENSE
COPY ./nuget.config ./ # so restore can see the Telerik feed

# runtime stage
RUN apt-get install -y --no-install-recommends \
  openssl libfontconfig1 libfreetype6 fontconfig fonts-noto-core \
  && fc-cache -f
ARG TELERIK_LICENSE
ENV TELERIK_LICENSE=$TELERIK_LICENSE

```

- `nuget.config` is copied *before* `dotnet restore`.
- Skia measures/rasterizes against **real font files** — QuestPDF ships its own, Telerik does not, and the base `aspnet` image has almost none. `fonts-noto-core` covers Latin; an Arabic naskh face (Amiri / Cairo / Noto Naskh Arabic) still needs adding before the container render of the Arabic voucher looks right.

- Build command gains: `--build-arg TELERIK_NUGET_KEY="$TELERIK_NUGET_KEY" --build-arg TELERIK_LICENSE="$(cat ~/.telerik/telerik-license.txt)"`

9.5 docker-compose.vm.yml

```
build:
  args:
    TELERIK_NUGET_KEY: ${TELERIK_NUGET_KEY:-}
    TELERIK_LICENSE:   ${TELERIK_LICENSE:-}
  environment:
    TELERIK_LICENSE:   ${TELERIK_LICENSE:-}    # runtime licence check parity
```

9.6 ~/.telerik/telerik-license.txt — not in the repo

Machine-level file (same one the KendoReact DevCraft upgrade created). `Telerik.Licensing` reads it automatically at build and runtime on a dev box. In Docker there is no such file, so the `TELERIK_LICENSE` env var (its *contents*) stands in.

10. The setup, as a bug journey

Each of these was hit, in order, bringing v2 up:

#	Symptom	Root cause	Fix
1	<code>NU1101: Unable to find package Telerik.Reporting</code> (restore)	Telerik packages aren't on nuget.org	added <code>nuget.config</code> with the private feed + <code>TELERIK_NUGET_KEY</code>
2	<code>CS0104: 'InvalidOperationException' is an ambiguous reference</code> (compile)	<code>API.Middleware</code> declares its own <code>InvalidOperationException</code>	qualified as <code>System.InvalidOperationException</code> (F8:35)
3	<code>CS0246: The type or namespace name 'LineShape' could not be found</code> (compile)	shape item types moved namespace between Reporting versions	replaced the header rule with a bottom-bordered <code>TextBox</code> (F9:93)
4	<code>DrawingFactoryUnavailableException: Cannot load assembly Telerik.Drawing.Skia</code> (runtime, first <code>Unit.Cm</code>)	<code>Telerik.Reporting</code> ships only the drawing abstraction	added <code>Telerik.Drawing.Skia</code> package
5	<code>ArgumentException: You can assign System.String, IImage, System.Drawing.Image, or SKBitmap instances only</code> (runtime, <code>PictureBox.Value</code>)	<code>PictureBox.Value</code> doesn't accept <code>byte[]</code>	<code>SKBitmap.Decode(logoPath)</code> (F9:71)
— works: <code>GET ../payment-report-v2/018037 → 200</code>			

11. Trace it yourself — where to put breakpoints

Break at	To watch
<code>PaymentReportsController.cs:43</code>	the incoming <code>paymentId</code> , <code>format</code>
<code>GeneratePaymentReportPdf.cs:71</code>	the EF query about to hit MySQL; inspect <code>result</code> after
<code>TelerikPaymentVoucherReportService.cs:28</code>	<code>data</code> fully populated, just before the report is built
<code>PaymentVoucherReport.cs:48</code>	the Telerik drawing backend loads on this line — step over to confirm no exception
<code>PaymentVoucherReport.cs:58</code>	after <code>BuildHeader</code> , before <code>BuildBody</code> — <code>report.Items[0]</code> has its children
<code>TelerikPaymentVoucherReportService.cs:34</code>	<code>result.HasErrors</code> / <code>result.DocumentBytes.Length</code>
<code>EditPaymentForm.tsx:126</code>	<code>arrayBuffer.byteLength</code> — the PDF size that came back

Frontend: DevTools → Network → the `payment-report-v2/...` request shows the exact URL, the `Authorization` / `Accept-Language` headers, status, and timing.